Healthcare & Clinics Mobile App for Patient Self Service

Project Documentation | Team ID 21, Section 03 | Database Systems Engineering and Distributed Backend Development (25CS1302E) | KLH Bachupally Campus, CSE | PBL 2026-27

Team members: K. Vishnu (2520030199), U. Vidhath (2520030597), K. Poojitha Sai (2520030103)

Guide: Dr. Spandana.S

1. Abstract

This project is a database-driven mobile self-service application for patients and clinics. Patients can book appointments, view their medical history, receive push alerts and file insurance claims from their phone. The system is built with an Android front end (Kotlin, Jetpack Compose), a Node.js REST API, a MySQL database and a Room DB offline cache. It aims to reduce paperwork and the administrative workload of clinic staff.

2. Problem Statement

Booking, record keeping and insurance claims in many clinics are still handled by phone calls, walk-ins and paper files. This causes long waiting times, low transparency for patients and a heavy workload for staff. There is no unified platform that connects patients, doctors, appointments, records, billing and claims in one place.

3. Objectives

1. Provide a self-service app for appointments, profile management and medical history.
2. Provide secure, scalable data management using MySQL and REST APIs, with offline access for patients.
3. Deliver real-time alerts through Firebase Cloud Messaging (FCM) and speed up insurance claim handling.

4. Scope

The system serves two user groups, patients and doctors. The modules in scope are:

- User profile management
- Appointment booking and management
- Medical history and records
- Insurance claims and billing
- Push notifications
- Background data synchronisation

5. Existing vs Proposed System

Feature	Existing System	Proposed System
Appointments	Phone calls and walk-ins	In-app booking with offline cache
Medical history	Limited, paper-based access	Digital history and reports on demand
Notifications	No real-time updates	FCM push alerts
Insurance claims	Slow, paper-based	Integrated claim and billing module
Limitation / gap	No unified platform; heavy staff dependency	One self-service platform: Android + Node.js + MySQL

6. Tools and Technologies

Layer	Technology	Purpose
Front end	Android (Kotlin, Jetpack Compose)	Patient and doctor user interface
Local cache	Room DB	Offline access to appointments and history
Back end	Node.js with REST APIs	Business logic, authentication and data access

Layer	Technology	Purpose
Database	MySQL	Persistent relational storage
Notifications	Firebase Cloud Messaging (FCM)	Real-time push alerts
Development tools	Android Studio, Git and GitHub, Postman	Coding, version control and API testing

7. System Architecture

The system follows a three-tier design.

1. **Android application (client tier):** Built with Kotlin and Jetpack Compose. A Room DB cache stores data locally so that key information stays available without a network connection.
2. **Node.js REST API (application tier):** Exposes the Authentication, Appointments, History and Claims services. The app talks to it over HTTPS using JSON.
3. **MySQL database (data tier):** Stores patients, doctors, appointments, medical records, claims and billing. The API reaches it through SQL queries.
4. **Firebase Cloud Messaging:** The back end triggers push alerts through FCM, for example appointment confirmations, reminders and claim status changes.

Data flow: the user acts in the app, the app calls the REST API over HTTPS/JSON, the API runs SQL against MySQL, and the response returns to the app, which also updates the Room cache.

8. Database Design

8.1 Entities

Entity	Primary key	Foreign keys	Other attributes
PATIENT	patient_id	none	name, dob, phone, email, address
DOCTOR	doctor_id	none	name, specialization

Entity	Primary key	Foreign keys	Other attributes
APPOINTMENT	appt_id	patient_id, doctor_id	date_time, status
MEDICAL_RECORD	record_id	patient_id, doctor_id	diagnosis, prescription
INSURANCE_CLAIM	claim_id	patient_id	amount, status
BILLING	bill_id	appt_id, claim_id	amount, paid_status

8.2 Relationships

- A patient can have many appointments (1 : N), and a doctor can have many appointments (1 : N).
- A patient can have many medical records (1 : N), and a doctor can write many medical records (1 : N).
- A patient can file many insurance claims (1 : N).
- Each appointment has one billing entry (1 : 1), and a claim can be linked to billing entries.

8.3 Design notes

- Foreign keys enforce referential integrity between patients, doctors, appointments, records, claims and billing.
- Indexes on frequently searched columns (such as patient_id and date_time) support quick record retrieval.
- Appointment, claim and payment status fields drive the notifications sent to users.

9. Module Description

Module	Description
Authentication and profile	Registration, login and profile management for patients and doctors.

Module	Description
Appointments	Patients choose a doctor and time slot, then view, reschedule or cancel bookings. Doctors see their schedule.
Medical history	Patients view diagnoses, prescriptions and reports on demand. Doctors add and update records.
Claims and billing	Patients file insurance claims and track their status. Billing links appointments to claims and payment status.
Notifications	FCM push alerts for booking confirmation, reminders and claim updates.
Offline sync	Room DB caches data locally and a background sync updates the server and the cache when the network returns.

10. Feasibility Study

- **Technical:** Kotlin, Jetpack Compose, Node.js and MySQL are well documented, and Android Studio and Postman are freely available.
- **Economic:** The stack is open source and uses the free tier of Firebase Cloud Messaging, so there is no licence cost.
- **Operational:** The patient interface is simple, routine work for clinic staff is reduced, and the system is easy to maintain.
- **Schedule:** The project is planned for completion within the PBL trimester timeline.

11. Testing and Results

- Planned checks cover CRUD operations, input validation and data synchronisation, tested with Postman.
- Indexed SQL queries are used for quick record retrieval.

12. Implementation Screenshots

The poster reserves two spaces for app screenshots. Add them here when the app is ready:

- Screenshot 1: Home / Appointments screen
- Screenshot 2: Medical History / Claims screen

13. Conclusion

The project provides a digital platform for appointments, medical history and insurance claims. It meets its objectives of less paperwork, more transparency and faster claim handling, using a single stack of Android, Node.js and MySQL.

14. Future Scope

- AI-based health recommendations
- Wearable (IoT) integration
- Telemedicine consultations
- Advanced health analytics

15. References

1. Android Developers: Jetpack Compose and Room, developer.android.com
2. Firebase Cloud Messaging documentation, firebase.google.com/docs/cloud-messaging
3. MySQL Reference Manual, dev.mysql.com/doc
4. Node.js Documentation, nodejs.org/docs